COURSE-BASED PROJECT DOCUMENTATION | DOC-19

Comprehensive Viva Q&A Master Guide

50+ Project-Specific Viva Questions, Answers, & Concise Examiner Phrases

Section A: Basic Project Questions

Q1: What is the primary purpose of this project?

Answer: It is a formal compiler front end that checks the grammatical validity of Java 21 source code and locates syntax errors to the exact line.

> **EXAMINER KEY PHRASE**
> Viva Bullet: "It validates Java 21 source code syntax using Flex and Bison and highlights errors in a Tkinter GUI."

Q2: Is this project a full Java compiler?

Answer: No. It is strictly a compiler front-end syntax checker. It does not perform semantic analysis, generate bytecode, or execute programs.

> **EXAMINER KEY PHRASE**
> Viva Bullet: "No, it is a syntax checker only; it does not perform semantic analysis, bytecode generation, or execution."

Q3: What version of Java is targeted and why?

Answer: Java 21 (LTS). It is the latest long-term support version that finalized modern syntax like record patterns, switch patterns, and sealed types.

> **EXAMINER KEY PHRASE**
> Viva Bullet: "Java 21 LTS because it contains finalized standard syntax for modern features like record patterns."

Section B: Compiler Design Concepts

Q4: What is a compiler front end?

Answer: The front end consists of lexical analysis, syntax analysis, and semantic analysis, converting source code into an intermediate representation.

> **EXAMINER KEY PHRASE**
> Viva Bullet: "The front-end phases process source code into an intermediate format independently of target machine code."

Q5: What is a Context-Free Grammar (CFG)?

Answer: A 4-tuple (V, Sigma, R, S) defining language syntax using recursive production rules that map nonterminals to strings of terminals.

> **EXAMINER KEY PHRASE**
> Viva Bullet: "A formal grammar whose production rules map a single nonterminal symbol to a string of terminals and nonterminals."

Section C: GNU Flex (Lexical Analysis)

Q6: Why choose GNU Flex?

Answer: Flex compiles regular expressions into high-speed table-driven DFAs in C, scanning tokens in linear O(n) time.

> **EXAMINER KEY PHRASE**
> Viva Bullet: "Flex generates fast, deterministic finite automata in C that tokenize source code in linear time."

Q7: What does YY_USER_ACTION do in Flex?

Answer: It is a macro executed before every matched token action to update yylloc line and column coordinates.

> **EXAMINER KEY PHRASE**
> Viva Bullet: "It is a Flex hook that synchronizes token line and column coordinates in yylloc before every action."

Section D: GNU Bison & LALR(1) Parsing

Q8: What is LALR(1) parsing?

Answer: Look-Ahead LR parsing with 1 token lookahead. It merges LR(1) states sharing identical core item sets to reduce table size.

> **EXAMINER KEY PHRASE**
> Viva Bullet: "LALR(1) is a bottom-up parsing method that merges identical LR(1) core states to create compact, fast parsing tables."

Q9: Why choose LALR(1) over Recursive Descent?

Answer: LALR(1) handles left-recursion naturally, eliminates recursive call stack overhead, and automates table generation from CFGs.

> **EXAMINER KEY PHRASE**
> Viva Bullet: "LALR(1) handles left-recursion naturally, avoids recursion overflow, and provides automated table generation."

Q10: How is the dangling-else ambiguity resolved?

Answer: By assigning higher precedence to ELSE over LOWER_THAN_ELSE, causing the parser to shift and bind 'else' to the innermost 'if'.

> **EXAMINER KEY PHRASE**
> Viva Bullet: "We assign ELSE higher precedence than LOWER_THAN_ELSE, causing the parser to shift and bind 'else' to the closest 'if'."

Section E: C Backend & Python ctypes

Q11: What are the return codes of check_syntax()?

Answer: 0 for success (valid syntax), 1 for syntax error, and 2 for file I/O or system error.

> **EXAMINER KEY PHRASE**
> Viva Bullet: "0 = Valid Syntax, 1 = Syntax Error, 2 = File I/O Error."

Q12: Why generate both a DLL and an EXE?

Answer: The DLL allows in-memory ctypes execution for the GUI, while the EXE provides a standalone CLI tool for scripts.

> **EXAMINER KEY PHRASE**
> Viva Bullet: "The DLL enables fast in-memory GUI execution via ctypes, while the EXE provides a standalone CLI tool."

Section F: Testing & Verification

Q13: How many automated tests were performed?

Answer: 73 automated tests in total: 59 parser tests in run_tests.py and 14 GUI tests in test_gui_backend.py, with a 100% pass rate.

> **EXAMINER KEY PHRASE**
> Viva Bullet: "73 automated tests in total: 59 parser tests and 14 GUI integration tests, all passed 100%."